

Java & Spark — All 30 MCQ Answer Guide

30 questions • All options • Correct option • Two-line explanation

Q1. A developer compiles a Java application on a Windows laptop, then the deployment team copies only the compiled .class file to a Linux server. Which component allows the same bytecode to execute on both operating systems?

- A. The source editor on each system
- B. The JVM available on each system
- C. The package statement on each system
- D. The JAR filename on each system

Correct option: B. The JVM available on each system

Explanation: Java bytecode is platform-independent, while the JVM is platform-specific. Each operating system provides a JVM that executes the same bytecode.

Q2. An HR application stores employee IDs in the exact order they were entered, and the same ID may appear more than once during testing. Which Java collection interface is the best starting choice?

- A. Map
- B. List
- C. Set
- D. Queue only

Correct option: B. List

Explanation: A List preserves insertion order and permits duplicate elements. A Set would remove duplicates, while a Map stores key-value pairs.

Q3. A trainee is building a score-processing program and wants the compiler to prevent non-integer values from being inserted into the score collection. Why is List safer than using a raw List?

- A. It provides compile-time element type checking
- B. It prevents duplicate scores automatically
- C. It sorts stored scores automatically
- D. It makes stored list elements immutable

Correct option: A. It provides compile-time element type checking

Explanation: Generics let the compiler enforce the element type at compile time. Therefore, List prevents incompatible values from being added accidentally.

Q4. A transport application declares a Vehicle reference, but at runtime it points to a Car object. The Car class overrides move(). When the application calls move() through the Vehicle reference, which method runs?

- A. The inherited Vehicle implementation
- B. No implementation runs through the reference
- C. The overridden Car implementation
- D. Both implementations run automatically together

Correct option: C. The overridden Car implementation

Explanation: Java uses dynamic method dispatch for overridden instance methods. The actual object type at runtime, Car, determines which implementation executes.

Q5. In a payroll application, the Manager constructor must send the employee ID to the Employee parent constructor before initializing manager-specific fields. Which call should the child constructor use?

- A. this(value) calls another constructor in the same class
- B. super() calls only the parent no-argument constructor
- C. this() calls the current-class no-argument constructor
- D. super(value) passes the value to a parent constructor

Correct option: D. super(value) passes the value to a parent constructor

Explanation: super(value) invokes the matching constructor of the parent class. It is used when the child constructor needs to initialize inherited state through the parent.

Q6. A developer is modelling a Car that uses an Engine object as one of its internal parts rather than making Car a subtype of Engine. Which code design most clearly represents composition?

- A. Order implements a Payable interface
- B. Order extends Payment to inherit its behavior
- C. Order contains a Payment object as a field
- D. Payment extends Order to reuse order behavior

Correct option: C. Order contains a Payment object as a field

Explanation: Composition represents a has-a relationship, where one object contains another object as a part. In the shown choices, containing a Payment object as a field demonstrates this pattern.

Q7. A document service must copy customer PDF files without changing any bytes in the file. Which Java I/O family is appropriate for this binary-copy requirement?

- A. Scanner and Formatter token/text utilities
- B. FileInputStream and FileOutputStream byte streams
- C. BufferedReader and PrintWriter text streams
- D. FileReader and FileWriter character streams

Correct option: B. FileInputStream and FileOutputStream byte streams

Explanation: PDF files are binary data, so byte streams are appropriate. FileInputStream and FileOutputStream copy raw bytes without applying character encoding.

Q8. A customer-registration system creates two Customer objects with the same customer ID. The class correctly overrides equals() and hashCode(), and both objects are added to a HashSet. What should the set normally do?

- A. Keep both because HashSet always compares object references only
- B. Treat them as the same logical element when equality and hash code match
- C. Sort both automatically before checking equality
- D. Keep both because equal hash codes always mean different objects

Correct option: B. Treat them as the same logical element when equality and hash code match

Explanation: HashSet uses hashCode() to locate a bucket and equals() to determine logical equality. If the objects are equal, the set normally stores only one of them.

Q9. A payment system has several unrelated classes that must all guarantee they provide the same set of operations, while each class can implement those operations differently. What is the main purpose of a Java interface in this design?

- A. Replace exception handling in an application
- B. Store every created object on heap memory
- C. Guarantee exactly one concrete implementation exists
- D. Define a contract implementing classes must provide

Correct option: D. Define a contract implementing classes must provide

Explanation: An interface defines a common contract without forcing one implementation. Different classes can implement the same operations in their own way.

Q10. A banking application must prevent other classes from directly changing an account balance and should expose controlled methods for updates. Which class design best demonstrates encapsulation?

- A. Allow callers to modify internal arrays directly
- B. Store unrelated data inside global variables directly
- C. Expose every field publicly to all callers
- D. Keep fields private and expose controlled methods

Correct option: D. Keep fields private and expose controlled methods

Explanation: Encapsulation hides internal state and controls access through methods. Private fields prevent direct modification while public methods can validate and manage updates.

Q11. A file-processing service opens a report file, reads its contents, and may throw an exception while processing. The developer wants the file resource to be closed reliably without repetitive cleanup code. Why is try-with-resources useful?

- A. It converts files into memory objects
- B. It removes checked file exceptions automatically
- C. It closes eligible file resources automatically
- D. It guarantees faster file access automatically

Correct option: C. It closes eligible file resources automatically

Explanation: Try-with-resources automatically calls close() on AutoCloseable resources. This cleanup happens even when an exception occurs during processing.

Q12. An e-commerce application has a stream of product prices. It must keep only prices above 500 and then double each remaining value for a simulation. Which Java stream expression performs the requirement?

- A. `prices.stream().collect(p -> p > 500).map(p -> p * 2)`
- B. `prices.stream().filter(p -> p > 500).collect().map(p -> p * 2)`
- C. `prices.stream().filter(p -> p > 500).map(p -> p * 2)`
- D. `prices.stream().map(p -> p * 2).filter(p -> p > 500)`

Correct option: C. `prices.stream().filter(p -> p > 500).map(p -> p * 2)`

Explanation: filter() removes values that do not satisfy the condition, and map() transforms the remaining values. Their order directly matches the requirement.

Q13. A Scala service classifies transaction-risk scores. A score of 5 is evaluated using 5 match { case x if x > 3 => "high"; case _ => "low" }. What result should the service produce?

- A. It returns the number '5'
- B. It returns the value 'low'
- C. It causes a compilation failure
- D. It returns the value 'high'

Correct option: D. It returns the value 'high'

Explanation: The score is 5, so the guard `x > 3` is true. Therefore the first matching case executes and returns "high".

Q14. A Spark job has an RDD of sales records and the developer wants to define another RDD by changing each record, without immediately producing a final result. Which Spark RDD operation is a transformation?

- A. `count()`
- B. `collect()`
- C. `first()`
- D. `filter()`

Correct option: D. `filter()`

Explanation: `filter()` is a Spark transformation and is evaluated lazily. `count()`, `collect()`, and `first()` are actions that trigger computation.

Q15. During a Spark job, one executor loses an RDD partition. Spark needs to recreate that partition instead of recomputing the entire application manually. Which information helps Spark rebuild it?

- A. The RDD lineage of transformations
- B. Only the executor memory that previously held the partition
- C. Only the notebook cell that first displayed the RDD
- D. Only the name of a temporary SQL view

Correct option: A. The RDD lineage of transformations

Explanation: RDD lineage records the transformations used to derive an RDD. Spark can use that lineage to recompute a lost partition from its dependencies.

Q16. A retail Spark application has a pair RDD containing (region, sale_amount) for every sale. The requirement is to calculate the total sales amount for each region. Which operation is suitable?

- A. `mapPartitions()` without a key
- B. `collect()` before grouping
- C. `reduceByKey(_ + _)`
- D. `repartition()` only

Correct option: C. `reduceByKey(_ + _)`

Explanation: `reduceByKey` combines values that share the same key. Using addition produces the total sale amount for every region.

Q17. A data engineer notices that a Spark DataFrame query is taking longer than expected. Before changing the code, the engineer wants to inspect how Spark plans to execute the query. Which tool is the best first check?

- A. Call `df.explain()` to inspect the plan
- B. Call `df.printSchema()` to inspect column types
- C. Call `df.show()` to display sample rows
- D. Call `df.count()` to count all rows

Correct option: A. Call `df.explain()` to inspect the plan

Explanation: `explain()` displays the logical and physical execution plans. This is useful for understanding joins, scans, exchanges, and other execution decisions before changing code.

Q18. A Spark transformation can be implemented either with an existing built-in function or with a newly written Python/Scala UDF. The team wants the option that Spark can optimize more effectively. Why is the built-in function usually preferred?

- A. A built-in function automatically removes all null values
- B. A built-in function guarantees that no shuffle can occur
- C. Spark can understand and optimize built-in expressions more easily
- D. A built-in function always executes only once per job

Correct option: C. Spark can understand and optimize built-in expressions more easily

Explanation: Spark understands built-in expressions as part of its query plan. This gives Catalyst and the execution engine more opportunities to optimize them than opaque UDF logic.

Q19. Two regional teams deliver CSV files for the same sales dataset, but schema inference reads one numeric column differently in the two files. Before the files are combined, what should the team do?

- A. Apply the same explicit schema when reading both files
- B. Union first and resolve incompatible column types only after failure
- C. Infer each file independently and union them without checking types
- D. Convert every column in both files to String regardless of meaning

Correct option: A. Apply the same explicit schema when reading both files

Explanation: An explicit common schema ensures consistent data types across files. This prevents type mismatches and makes the later union predictable.

Q20. A data engineer receives large tabular data with named columns, built-in aggregations, SQL expressions, and Spark optimizations. Which data abstraction is more suitable?

- A. RDD with low-level functions only
- B. A text-file iterator with no structured column metadata
- C. A DataFrame with a defined schema and SQL expressions
- D. A local Scala collection processed only on the driver

Correct option: C. A DataFrame with a defined schema and SQL expressions

Explanation: DataFrames provide structured columns and a schema while allowing Spark SQL optimizations. They are well suited to structured tabular analytics.

Q21. A stateful Structured Streaming application processes account events. After a driver failure, the application must restart and continue using previously stored progress and state instead of starting from the beginning. What should be configured?

- A. Cache the query result only in executor memory
- B. Create a temporary SQL view for the stream
- C. Repartition the streaming DataFrame on every trigger
- D. Configure a durable checkpoint location

Correct option: D. Configure a durable checkpoint location

Explanation: A durable checkpoint stores streaming progress and state needed for recovery. It allows the query to restart without losing its previously recorded progress.

Q22. A live delivery dashboard calculates metrics in 10-minute windows. Business users still want events accepted when they arrive up to 2 minutes late because of network delay. Which approach is appropriate?

- A. Use a processing-time window with no event-time handling
- B. Use only a processing-time window with no late-data control
- C. Use an event-time window but no watermark for late-data control
- D. Use event-time windows with a suitable watermark

Correct option: D. Use event-time windows with a suitable watermark

Explanation: Event time aligns windows with when events actually occurred, while a watermark defines how long late data can be considered. Together they support controlled late-event handling.

Q23. Two worker threads process inventory updates. Thread 1 holds Lock 1 and waits for Lock 2, while Thread 2 holds Lock 2 and waits for Lock 1. Neither thread can proceed. What concurrency problem has occurred?

- A. Race condition
- B. Deadlock
- C. Thread starvation
- D. Livelock

Correct option: B. Deadlock

Explanation: A deadlock occurs when threads hold resources while waiting for resources held by each other. Neither thread can make progress until the cycle is broken.

Q24. A banking program updates both an accounts table and an audit table as one business operation. If either update fails, neither table should retain the change. Which JDBC approach is appropriate?

- A. Use separate connections with auto-commit enabled for each update
- B. Commit after updating the first table, then continue
- C. Skip exception handling and commit each statement separately
- D. Disable auto-commit, run both updates, then commit or roll back

Correct option: D. Disable auto-commit, run both updates, then commit or roll back

Explanation: A transaction groups both updates into one atomic unit. With auto-commit disabled, the program can commit only if both succeed or roll back when either fails.

Q25. A team is starting a new Spark real-time project and must choose between the older DStream API and the newer Structured Streaming model. Which statement is most appropriate for the new implementation?

- A. DStreams and Structured Streaming expose exactly the same programming model
- B. Structured Streaming is the newer DataFrame-based streaming model
- C. Structured Streaming cannot support event-time operations
- D. DStreams are the newer DataFrame-based streaming model

Correct option: B. Structured Streaming is the newer DataFrame-based streaming model

Explanation: Structured Streaming uses the structured DataFrame/Dataset programming model. It is the newer Spark streaming API and supports features such as event-time processing.

Q26. A partner system drops new CSV files into a landing directory throughout the business day. The data engineering team wants Spark to continuously process newly arriving files as streaming input. What can Structured Streaming do?

- A. Require a socket source before any file can be processed
- B. Treat newly arriving files as incremental streaming input
- C. Support only Parquet files and not CSV file streams
- D. Read the directory only once and ignore files that arrive later

Correct option: B. Treat newly arriving files as incremental streaming input

Explanation: Structured Streaming can use a directory as a file source and process newly arriving files incrementally. CSV is a supported file format when configured appropriately.

Q27. A multi-threaded Java service stores session information in a shared map that is updated by several threads at the same time. At a conceptual level, why might ConcurrentHashMap be chosen over HashMap?

- A. It prevents every possible application concurrency bug
- B. It guarantees all keys remain sorted automatically
- C. It automatically opens and manages database connections
- D. It supports safer access from multiple concurrent threads

Correct option: D. It supports safer access from multiple concurrent threads

Explanation: ConcurrentHashMap is designed for concurrent access and provides thread-safe map operations. A normal HashMap is not designed for unsynchronized concurrent updates.

Q28. A Java application starts a time-consuming remote calculation but should continue doing other work and process the result later when it becomes available. What does CompletableFuture help represent?

- A. It represents a database transaction definition
- B. It represents an asynchronous result completed later
- C. It represents a file lock held permanently
- D. It represents the JVM garbage-collection process

Correct option: B. It represents an asynchronous result completed later

Explanation: CompletableFuture represents a result that may be completed asynchronously. It also supports composing follow-up actions without blocking the initiating thread.

Q29. Two threads in an order service both increment the same shared counter. After heavy concurrent activity, the final count is lower than expected because some updates were overwritten. Which concept best explains the problem?

- A. A race condition caused by unsynchronized shared updates
- B. Thread starvation caused by scheduling priority
- C. A livelock where threads keep reacting without making progress
- D. A deadlock between two locks

Correct option: A. A race condition caused by unsynchronized shared updates

Explanation: The threads can read and update the shared value at overlapping times, causing lost updates. This timing-dependent behavior is a race condition.

Q30. A JDBC money-transfer operation successfully debits the source account, but the subsequent credit to the destination account fails. The business rule requires the transfer to be all-or-nothing. What should the program do?

- A. Commit the debit even though the credit failed
- B. Ignore the failed transaction and retry only the credit separately
- C. Roll back the entire database transaction
- D. Enable auto-commit and continue with the partial update

Correct option: C. Roll back the entire database transaction

Explanation: An all-or-nothing transfer requires atomicity. If the credit fails, the earlier debit must be rolled back so the database is not left partially updated.